Ders 17 Genişletilmiş Notu: Uygulama: Arayüzler, İstisnalar, Jenerikler ve Koleksiyonlar

Ders 17 Genişletilmiş Notu: Uygulama: Arayüzler, İstisnalar, Jenerikler ve Koleksiyonlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Arayüzler (Interface)
- 2. Örnek: Kitap ve Dörtgen İçin Ortak Karşılaştırma Arayüzü
- 3. İstisnalar (Exception)
- 4. Jenerikler (Generics)
- 5. Koleksiyonlar (Collections)
- 6. Örnek: Negatif İndeks Destekleyen MyArrayList
- 7. MyArrayList'e Silinen Elemanları Kaydetmek

Bu oturum bir uygulama dersidir. Hoca dört konuyu "ufak ufak göstererek ve bir şeyler yazarak" geçti: arayüzler, istisnalar, jenerikler ve koleksiyonlar. Örnekleri aynı yerde görülsün diye tek bir test sınıfının yanına yazdı. Bu yüzden aşağıdaki kodlarda sınıflar tek dosyada durur.

1. Arayüzler (Interface)

1.1 Tanım ve İçerik

Sınıfları `public class SinifAdi` diye tanımlıyorduk. Arayüzler de birebir aynı biçimde tanımlanır, yalnız `class` yerine `interface` yazılır. Hoca örnekte adını `MyInterface` koydu.

Arayüzün asıl işi, soyut sınıflarda olduğu gibi içine metotlar yazmaktır. Metodun soyut olduğu ayrıca belirtilmez. Yalnızca imzası yazıldığı için zaten soyut metot gibi davranır. Metotlar parametre de alabilir. Arayüz yazmak, aslında böyle metotları alt alta sıralamaktır.

Arayüzde başka şeyler de bulunabilir:

- **Değişkenler:** Arayüze değişken de yazılabilir, ama çok kullanılmaz. Bunlar sabit olur. `final` yazılmasa da bir daha değiştirilemez.
- **`default` metotlar:** Gövdesi olan metotlardır. Soyut olmadıkları için uygulayan sınıf yazmasa da çağrılabilirler. Hoca bunun çok kullanılmadığını söyledi.
- **`static` metotlar:** Arayüzde statik metot da olabilir.

En sık karşılaşılan kullanım, yalnızca metot imzalarını sıralamaktır.

1.2 Uygulama: implements

Kalıtımda `extends` yazıyorduk. Arayüzü uygulamak için `implements` yazılır ve arkasına arayüzün adı gelir. Bu satır derleyiciye "buradaki metotları yazmak zorundasın" der.

`extends` ile arasında önemli bir fark vardır. `extends` yalnızca bir kere yazılabilir, ama `implements` ile birden fazla arayüz uygulanabilir. Bu sayı iki ile sınırlı değildir, 3, 5 ya da 10 olabilir. Arayüzler de birbirinden kalıtım alabilir.

Hoca slayttaki örneği çalıştırarak hangi sınıfta hangi metodun çağrıldığını gösterdi. Alt sınıf `method1` metodunu kendisi uyguladığı için, çağrı alt sınıftaki `method1` metoduna gider. `default` metot ise alt sınıfta yazılmasa da çağrılır.

Aşağıdaki kod, bu anlatımı derlenebilir hale getirir. Slaytın aynısı değildir; slayttaki kod altyazıda okunmuyor.

```
interface MyInterface {
    String SABIT = "değiştirilemez";

    public void method1(String mesaj);
    public void method2();

    default void varsayilanMetot() {
        System.out.println("MyInterface içindeki default metot çalıştı");
    }

    static void statikMetot() {
        System.out.println("MyInterface içindeki static metot çalıştı");
    }
}

interface DigerArayuz {
    public void method3();
}

interface BirlesikArayuz extends MyInterface, DigerArayuz {
}

class Child implements BirlesikArayuz {
    @Override
    public void method1(String mesaj) {
        System.out.println("Child sınıfındaki method1 çalıştı: " + mesaj);
    }

    @Override
    public void method2() {
        System.out.println("Child sınıfındaki method2 çalıştı");
    }

    @Override
    public void method3() {
        System.out.println("Child sınıfındaki method3 çalıştı");
    }
}

public class ArayuzDeneme {
    public static void main(String[] args) {
        Child c = new Child();
        c.method1(MyInterface.SABIT);
        c.method2();
        c.method3();
        c.varsayilanMetot();
        MyInterface.statikMetot();
    }
}
```

Çıktı (javac ile denendi):

```
Child sınıfındaki method1 çalıştı: değiştirilemez
Child sınıfındaki method2 çalıştı
Child sınıfındaki method3 çalıştı
```

```
MyInterface içindeki default metot çalıştı
MyInterface içindeki static metot çalıştı
```

Hocanın söylediği iki kural javac ile denendi. Soyut metodu yazmayan sınıf derlenmez (`C is not abstract and does not override abstract method m() in I`). Arayüzdeki değişkene atama yapan kod da derlenmez (`cannot assign a value to static final variable S`).

1.3 Arayüz Neden Var?

Akla şu soru gelir: Metodun içini zaten sınıfta yazıyoruz, o zaman arayüz diye bir şey niye var? Hoca kendisine mantıklı gelen iki sebep söyledi:

- **Çoklu kalıtım:** `extends` bir kere yazılabildiği, arayüzde ise birden fazla uygulama yapılabildiği için çoklu kalıtım uygulanabilir.
- **Daha iyi tasarım:** Bir sonraki örnek bununla ilgilidir.

  Ders sonrası not: Hoca "çoklu kalıtım uygulanabilir" dedi. Bu yalnızca metot sözleşmeleri için geçerlidir. Arayüzdeki alanlar sabittir (javac ile denendi), yani birden fazla arayüzden örnek alanı (nesne durumu) devralınmaz. Bu yüzden "çoklu kalıtıma benzer bir yapı" demek daha doğrudur.

2. Örnek: Kitap ve Dörtgen İçin Ortak Karşılaştırma Arayüzü

2.1 İki İlgisiz Sınıf

Hoca iki basit sınıf yazdı. Kitap sınıfının (`Book`) iki üye alanı ve fiyatı veren bir `getPrice` metodu vardır. Dörtgen sınıfının (`Rectangle`) kenarları ve alanı veren, `double` döndüren bir `getArea` metodu vardır. Alan `x * y` ile hesaplanır.

Bu iki sınıfın aslında hiç alakası yoktur, ortak değişkenleri de yoktur. Yine de şöyle bir bağlantı kurulabilir: İkisinde de büyüklük-küçüklük ilişkisi kurulabilir. İngilizcesiyle "comparable". Hoca bunun için `Comparable` adlı bir arayüz yazdı ve içine `isBiggerThan` metodunu koydu. Bu metodun, `String` sınıfındaki `compareTo` metoduna benzediğini söyledi.

2.2 Metodun Yazılışı

Metot parametre olarak arayüz tipinde bir nesne alır. Gelen nesne önce sınıfın kendi tipine dönüştürülür (cast), sonra sınıfın ölçütü karşılaştırılır. Kitap için ölçüt fiyattır. Metot, nesne büyükse 1, eşitse 0, küçükse -1 döndürür. Aynı metot dörtgende alana göre yazılır.

```
interface Comparable {
    public int isBiggerThan(Comparable c);
}

class Book implements Comparable {
    private String name;
    private double price;

    public Book(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }

    @Override
    public int isBiggerThan(Comparable c) {
        Book other = (Book) c;
        if (getPrice() > other.getPrice()) return 1;
        else if (getPrice() == other.getPrice()) return 0;
```

```
        else return -1;
    }
}

class Rectangle implements Comparable {
    private double x;
    private double y;

    public Rectangle(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getArea() {
        return x * y;
    }

    @Override
    public int isBiggerThan(Comparable c) {
        Rectangle other = (Rectangle) c;
        if (getArea() > other.getArea()) return 1;
        else if (getArea() == other.getArea()) return 0;
        else return -1;
    }
}
```

Hoca `Book` içindeki metodu `Rectangle` sınıfına kopyaladı ve `Book` adlarını `Rectangle`, `getPrice` çağrılarını `getArea` yaptı. Kopyalamada bir yer gözden kaçtı:

Soru: Bir tane daha `Book` kalmış hocam. `Book other = (Rectangle)` yazmışız. **Cevap:** Hoca o satırı da `Rectangle` olarak düzeltti.

2.3 Çalıştırma

Hoca iki dörtgen oluşturup `isBiggerThan` metodunu çağırdı. Alanı daha büyük olan ikinci dörtgen için sonuç "büyük" çıktı. Eşit alanlı dörtgenlerde 0 dönmesini bekliyordu, ama -1 döndü. Sebebi, ikinci koşulda büyüktür işaretinin değiştirilmemiş olmasıydı.

```
// Derste yazılan hatalı sürüm
if (getArea() > other.getArea()) return 1;
else if (getArea() > other.getArea()) return 0;  // Burada == olmalıydı
else return -1;
```

İki değer eşitse iki koşul da yanlıştır, akış `else` kısmına gider. Bu yüzden -1 döner. Hatalı sürüm eşit değerlerle javac ile denendi, -1 döndürdü. Yukarıdaki düzeltilmiş sürüm eşit alanlarda 0, büyük alanda 1, küçük alanda -1 döndürdü.

2.4 Tasarım Kazancı

Hoca, sınıfların alakasız olmasının sorun olmadığını bir senaryoyla açıkladı. Takım baştan şu kararı verir: Birbiriyle büyüklük-küçüklük ilişkisi kurulabilen nesneler `Comparable` arayüzünü uygulasın. Bu sayede sınıfın içine hiç bakmadan, yalnızca `implements Comparable` satırına bakarız. `Book` sınıfından bir nesne oluşturup `isBiggerThan` metodunu çağırabileceğimizi biliriz. Yüz sınıfımız olduğunu ve yirmisinde bu arayüzün uygulandığını düşünün. Hangilerinin karşılaştırılabildiği bu tek satırdan anlaşılır.

Ölçüt sınıftan sınıfa değişir. Kitapta fiyat, dörtgende alan, başka bir şeyde başka bir şey olabilir. Ama soyutlama aynıdır. Büyüklük-küçüklük ilişkisi kurulamayan sınıflar da olabilir. Örneğin kırmızı, siyah, beyaz gibi renkleri tutan bir sınıf bu arayüzü uygulamaz.

> **Ek bilgi (derste anlatılmadı):** Java'nın kendi `java.lang.Comparable<T>` arayüzü vardır ve metodu `compareTo` metodudur. Derste aynı adla ayrı bir arayüz yazıldı; aynı dosyada tanımlandığı için bu ad o dosyada kendi arayüzümüzü gösterir. Bu kod

derlendi. Ayrıca metodu `Book` nesnesi üzerinde `Rectangle` parametresiyle çağırmak derlenir, ama çalışırken `ClassCastException` verir (javac ile denendi). Yani arayüz ortak olsa da karşılaştırma aynı türden nesneler için anlamlıdır.

3. İstisnalar (Exception)

3.1 Neden Gerekli?

Program çalışırken beklenmeyen durumlar olur. Bir dosya okumak isteriz, dosya yoktur ya da dosyayı açma yetkimiz yoktur. Bir algoritmamız vardır, bin adımın 178. adımında sıfıra bölme olur. Ya da dizinin olmayan bir indisine erişiriz. Böyle durumların bir şekilde kontrol edilmesi gerekir. Program orada hata verip kapanabilir, ya da biz duruma uygun bir şey yapabiliriz. İstisnalar bunun içindir.

3.2 try ve catch

İlk iki anahtar sözcük `try` ve `catch`tir. İstisna olabilecek yer `try` bloğuna alınır. İstisna olursa ne yapılacağı `catch` bloğuna yazılır. Hocanın örneğinde iki `catch` vardı: biri sıfıra bölme hatası, diğeri dizinin olmayan indeksine erişme için.

Akış şöyledir:

1. `try` bloğu çalışmaya başlar.
2. Bir satırda istisna oluşursa o satırın altında kalan satırlar çalışmaz.
3. `catch` blokları aranır. İstisna hangisine uyuyorsa o `catch` çalışır.
4. Akış `try-catch` yapısından sonra devam eder.

Hoca örneği iki kez çalıştırdı. İlkinde `a` değişkeninin içinde 1 vardı ve sıfıra bölme hatası alındı. Hata burada alındığı için diğer `catch` bloğuna girilmedi. Bu durumda `a` 0 yapıldı, 5 ile çarpılınca yine 0 kaldı. Sonunda `a`'nın son değeri 0, `b`'nin son değeri de 0'ın 10 katı yazıldığı için 0 oldu. İkinci çalıştırmada hata öteki satırda alındı, bu kez diğer `catch` bloğuna girildi. O sırada `a`'nın değeri 5'ti, `catch` içinde karesi alındı ve son değer 25 oldu. Bu sayılar python ile hesaplandı: $0 \times 5 = 0$, $0 \times 10 = 0$, $5 \times 5 = 25$.

Hocanın vardığı sonuç şudur:

> **Hocanın vurgusu:** Burada anlamamız gereken, istisna olabilecek şeyi `try` içine, hangi istisnada ne yapılacağını `catch` bloklarına yazmaktır. `catch` içinde durum düzeltilir, programdan çıkılır, dosya kapatılır, işimiz neyse o yapılır. Program akışı buna göre devam eder.
>
> **Ders sonrası not:** Hoca bir yerde "`finally` olmadığı için program burada bitiyor" dedi, sonra da akışın devam ettiğini söyledi. Doğrusu ikincisidir. Uygun `catch` çalıştıktan sonra program `try-catch` yapısının altından devam eder. Bir sonraki koddaki son satır her çalıştırmada yazıldı (javac ile denendi).

3.3 finally ve throw

`finally` bloğu, hangi `catch` bloğuna girilirse girilsin sonunda çalışır. Hoca bunun aslında tek farkın bu olduğunu, eklenip eklenmemesinin bize kalmış olduğunu söyledi.

Kendimiz de doğrudan istisna oluşturabiliriz: `throw new Exception(...)`. Böyle bir hata olduğunu bildirip programın sonlanmasını sağlayabiliriz. Hoca, sınıf içindeki bir metotta istisna tanımlama örneğini de gösterdi, ama bu ayrıntının çok önemli olmadığını söyleyerek geçti.

Aşağıdaki kod, sıfıra bölme ve dizi indeksi hatalarını `finally` ile birlikte gösterir. Slaytın aynısı değildir, ama kısa nottaki iskeleti tam programa çevirir.

```
public class IstisnaDeneme {
    static void calistir(int bolen, int indeks) {
        int a = 1;
        int[] d = new int[3];
        try {
            int b = a / bolen;
```

```
            d[indeks] = b;
            System.out.println("Hata çıkmadı");
        } catch (ArithmeticException e) {
            System.out.println("Sıfıra bölme");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Dizi sınırı aşıldı");
        } finally {
            System.out.println("finally her durumda çalışır");
        }
        System.out.println("Program akışı devam ediyor");
    }

    public static void main(String[] args) {
        calistir(0, 0);
        calistir(1, 5);
        calistir(1, 0);
    }
}
```

Çıktı (javac ile denendi):

```
Sıfıra bölme
finally her durumda çalışır
Program akışı devam ediyor
Dizi sınırı aşıldı
finally her durumda çalışır
Program akışı devam ediyor
Hata çıkmadı
finally her durumda çalışır
Program akışı devam ediyor
```

Birinci çağrıda `d[indeks] = b` satırı çalışmaz, çünkü üstündeki satır istisna fırlatmıştır.

> **Ek bilgi (derste anlatılmadı):** `catch` blokları yukarıdan aşağıya denenir, bu yüzden özel istisna genel olandan önce yazılır. `catch (Exception e)` bloğunu `catch (ArithmeticException e)` bloğundan önce yazan kod derlenmez (javac ile denendi: `exception ArithmeticException has already been caught`).
>
> **Ek bilgi (derste anlatılmadı):** Dersteki "check exception" ifadesi, derleyicinin denetlediği istisnalarla ilgilidir. Bir metot `throw new Exception("...")` yazıp bunu yakalamıyorsa imzasına `throws Exception` eklenmelidir. Eklenmezse kod derlenmez (javac ile denendi: `unreported exception Exception; must be caught or declared to be thrown`).

4. Jenerikler (Generics)

4.1 Jenerik Metot

Metot yazarken dönüş tipini ve parametre tipini belirtiriz: Bu metot `double` döndürür, şu metot `String` parametre alır. Böyle yazılan metot yalnızca o tiple çalışır. Hangi tipi verirsek verelim çalışsın istiyorsak jenerikleri kullanırız.

Hocanın slayttaki örneği, bir dizinin ya da listenin elemanlarını yazdıran bir metottu. Elemanların tipi bu iş için önemsizdir. Bu yüzden metoda bir tip parametresi <T> eklenir ve tip yazılan yerlere `T` yazılır. Böyle yazıldıktan sonra `Integer`, `Double` ya da `Character` listesi göndersek de aynı kod çalışır. Faydası şudur: Her tip için ayrı metot yazmak yerine, hepsi için çalışan tek bir genel metot yazarız.

4.2 Jenerik Sınıf

Aynı şey sınıflar için de yapılır. Sınıf adının yanına tip parametreleri yazılır. Hoca aynı sınıfı bir kez `String` ve `Integer` için, bir kez de iki karakter için oluşturdu; bu sınıfın iki tip parametresi vardı. Sınıf, oluşturulurken verilen tipe göre çalışır. Hocanın senaryosu şuydu: Fiyat bazen

double, bazen integer olabilir. Sınıfı jenerik yaparsak kullanan kişi double tanımlarsa double, integer tanımlarsa integer olarak çalışır. Slayttaki kod da yalnızca değerleri yazdırıyordu.

```
import java.util.List;

class Cift<T, U> {
    private T birinci;
    private U ikinci;

    public Cift(T birinci, U ikinci) {
        this.birinci = birinci;
        this.ikinci = ikinci;
    }

    public void yazdir() {
        System.out.println(birinci + " " + ikinci);
    }
}

public class JenerikDeneme {
    public static <T> void yazdir(List<T> liste) {
        for (T eleman : liste) {
            System.out.print(eleman + " ");
        }
        System.out.println();
    }
}
```

Kullanımda tip, sınıf adının yanında verilir: `Cift<String, Integer>` ya da `Cift<Character, Character>`. Metot `Integer`, `Double` ve `Character` listeleriyle, sınıf bu iki tip çiftiyle çalıştırıldı (javac ile denendi).

> **Ders sonrası not:** Hoca "hangi tiple çalışacağı runtime'da belirleniyor" dedi. Doğrusu, tip denetimi derleme zamanında yapılır. `List<Integer>` listesine `String` eklemek derlenmez (javac ile denendi: `String cannot be converted to Integer`). Çalışma zamanında ise tip bilgisi ayrı tutulmaz: `new ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()` ifadesi `true` verir (javac ile denendi).

5. Koleksiyonlar (Collections)

5.1 Hiyerarşi: Veri Yapıları da Nesneye Yöneliktir

Hoca koleksiyon hiyerarşisinin şemasını açtı. Java'daki `ArrayList`, `Vector` ve benzeri yapılar da nesneye dayalı kavramlarla kurulmuştur. Şema üstten alta şöyle okunur:

1. En üstte `Iterable` vardır.
2. Onun altında `Collection` vardır. Bütün yapılar onun metotlarını uygular.
3. `Collection` altında `List` vardır. Şemada kesik çizgiyle gösterildiği için `List` bir arayüzdür. Şema hangi bağın `extends`, hangisinin `implements` olduğunu da gösterir.
4. `List` altında `ArrayList`, `LinkedList` ve `Vector` gibi somut sınıflar yer alır.

Hoca, IDE'de `List` arayüzünü açıp gösterdi. İçinde, bizim üstte yazdığımız arayüzdeki gibi metotlar vardır. Kendimiz bir liste yazarsak bu metotları tek tek uygulamamız gerekir. Yapı, ders başında kurduğumuz yapının aynısıdır.

Bunun faydası şudur: Yapıların hepsi aynı arayüzü uyguladığı için hepsinde `add`, `remove`, `size` gibi aynı metotlar vardır ve aynı şekilde çalışırlar.

> **Ders sonrası not:** Hoca hiyerarşiyi anlatırken `HashMap` yapısını da aynı gruba koyup "hepsi bir `Collection`" dedi. `HashMap` bir `Collection` değildir, `Map` arayüzünü uygular. `Collection<String> c = new HashMap<String, String>();` satırı derlenmez ve `Collection.class.isAssignableFrom(HashMap.class)` ifadesi `false` verir (javac ile denendi). `ArrayList` için aynı ifade `Collection` ve `Iterable` ile `true` verir.

5.2 ArrayList ve LinkedList

Dizide boyutu sabit tanımlarız: 20 elemanlı, 100 elemanlı gibi. C programlamada hep böyle yaparız. `ArrayList` yapısının faydası dinamik olmasıdır, boyutu değişebilir.

Bağlı liste (linked list) veri yapılarında C ile yazılmış olmalı. Burada doğrudan 23. elemana erişilemez, ileri ileri ya da geri geri gidilir. Hoca bunun kullanım alanı olarak IDE'deki geri al ve ileri al işlemlerini verdi. Durumları bağlı liste içinde tutarsak bir adımda ya bir ileri ya da bir geri gideriz.

İki yapının bellekteki farkı da şudur: Dizide elemanlar alt alta, sıralı durmak zorundadır. Bağlı listede ise araya boşluklar girebilir, çünkü bir yerden öbür yere bağlantı vardır. Hoca bunu şu benzetmeyle anlattı: C dizininde 500 MB, D dizininde 500 MB yer var. 700 MB'lık bir şeyi tek parça olarak atayamayız, biraz oraya biraz buraya koymamız gerekir. Dizide bunu yapamazken bağlı listede yapabiliriz. Hoca bunun işletim sistemlerinde daha ayrıntılı görüleceğini söyledi.

> **Ders sonrası not:** Altyazıda "500 MW" geçiyor, doğrusu büyük olasılıkla MB'dir. Benzetme büyük olasılıkla aynı disk ya da bellekteki parçalı boş alanlar için daha doğrudur; dosya sistemleri bir dosyayı büyük olasılıkla iki ayrı sürücüye bölmez. Ayrıca Java'nın `LinkedList` sınıfında `get(i)` metodu vardır ve çalışır (javac ile denendi). Bu metot büyük olasılıkla düğümleri yine sırayla gezer, yani hocanın anlattığı gibi "doğrudan" erişim değildir.

5.3 ArrayList Kullanımı ve Iterator

`ArrayList` de aslında jeneriktir. Tip yerine `String` yazarsak `String` listesi oluşur. Elemanlar `add` metoduyla eklenir. Slaytta ayrıca `addAll` metodu da vardı. `remove` metoduna tamsayı verilirse o sıradaki eleman silinir, nesne verilirse o nesne silinir. Hoca bu örnekte 3. sıradakini silmeyi söyledi. `retainAll` ise verilen koleksiyondakileri tutar, kümelerdeki kesişime benzer.

Yazdırma için önceden bir `for` döngüsü kuruyorduk. Bir `iterator` tanımlayıp, devamı varsa `next` diyerek de yazdırabiliriz. Hoca buna soru sordu: `for` varken bunu niye yapalım? Çünkü algoritmamız her zaman bir sonrakine gitmeyi gerektirmeyebilir. İki ileri, bir geri, üç ileri gitmemiz gerekebilir. `previous` ile bu mantığı kurabiliriz. Sonraki örnekte hem `hasNext` hem de `hasPrevious` kullanıldı.

> **Ders sonrası not:** Hoca bunu `Iterator` üzerinden anlattı. `Iterator` yalnızca ileri gider. `Iterator<String>` üzerinde `previous()` çağrısı derlenmez (javac ile denendi: `cannot find symbol`). Geri gitmek için `ListIterator` kullanılır.

`LinkedList` örneği de `ArrayList` örneğinin aynısıdır. `ArrayList` yerine `LinkedList` yazıldığında kod aynen çalışır.

Aşağıdaki kod bu metotları gösterir. Slaytın aynısı değildir; slayttaki kod altyazıda okunmuyor.

```java
import java.util.ArrayList;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.ListIterator;

public class KoleksiyonDeneme {
    public static void main(String[] args) {
        ArrayList<String> liste = new ArrayList<>();
        liste.add("A");
        liste.add("B");
        liste.add("C");
        liste.add("D");
        liste.add("E");
        System.out.println(liste.size());

        liste.remove(3);    // 3. sıradaki eleman silinir: D
        liste.remove("A");  // A nesnesi silinir
        System.out.println(liste);
```

```
        Iterator<String> it = liste.iterator();
        while (it.hasNext()) {
            System.out.print(it.next() + " ");
        }
        System.out.println();

        ListIterator<String> li = liste.listIterator();
        System.out.print(li.next() + " ");
        System.out.print(li.next() + " ");
        System.out.println(li.previous());

        ArrayList<String> diger = new ArrayList<>();
        diger.add("C");
        diger.add("E");
        diger.add("Z");
        liste.retainAll(diger);
        System.out.println(liste);

        List<String> bagli = new LinkedList<>(liste);
        bagli.add("F");
        System.out.println(bagli + " " + bagli.size());
    }
}
```

Çıktı (javac ile denendi):

```
5
[B, C, E]
B C E
B C C
[C, E]
[C, E, F] 3
```

5.4 HashMap

> **Hocanın vurgusu:** Burada aslında `HashMap` önemlidir. Çok iyi bir veri yapısıdır. Python'da dictionary olarak geçer. $O(1)$'de arama yapmamızı sağlar.

Diğer yapılarda bir eleman eklenirken burada iki şey eklenir: biri anahtar (key), diğeri değer (value). Hocanın örneğinde anahtarlar sayıydı, değerler adlardı. 5 deyince bir adı, 1 deyince başka bir adı verir. `ArrayList` yapısında olduğu gibi `get` ve `size` metotları vardır. 17 anahtarındaki adı başka bir adla değiştirebiliriz. Doğrudan `put` ile aynı anahtara yeni değer verdiğimizde de değer değişir. `remove` ile 5 anahtarı silinir. `clear` hepsini siler. Hoca bu örneğin sonunda `null` yazdığını gösterdi.

```
import java.util.HashMap;

public class HashMapDeneme {
    public static void main(String[] args) {
        HashMap<Integer, String> harita = new HashMap<>();
        harita.put(5, "ad1");
        harita.put(1, "ad2");
        harita.put(17, "eskiAd");
        System.out.println(harita.get(5) + " " + harita.size());

        harita.put(17, "yeniAd");
        System.out.println(harita.get(17));

        harita.remove(5);
        System.out.println(harita.get(5));
```

```
        harita.clear();
        System.out.println(harita.size() + " " + harita.get(17));
    }
}
```

Çıktı (javac ile denendi):

```
ad1 3
yeniAd
null
0 null
```

> **Ders sonrası not:** `HashMap` aramasının $O(1)$ olması büyük olasılıkla ortalama durum içindir. Çok çakışma olursa arama büyük olasılıkla yavaşlar.

6. Örnek: Negatif İndeks Destekleyen MyArrayList

6.1 Fikir

Hoca son örnekte bir noktayı göstermek istedi: Bu veri yapılarını içlerini bilmeden kullanıp yeni bir şeyler oluşturabiliriz. `ArrayList` sınıfından kalıtım alan jenerik bir sınıf yazdı: `class MyArrayList<E> extends ArrayList<E>`. Herhangi bir tip alabilmesi için `<E>` yazdı ve `ArrayList` sınıfını import etti.

Çalışan, `ArrayList` kullanan bir kodunuz olduğunu düşünün. Her yerinde `ArrayList` adını `MyArrayList` yapsanız kod yine aynı şekilde çalışır. Peki niye `extends` yaptık? Bazı davranışları değiştirmek istiyoruz. Kalıtım zaten böyle çalışır: Bütün özellikleri alır, kendi eklemek istediğini ekler. Hoca "çalışan, öğrenci, insandır, ondan kalıtım alır" örneğini hatırlattı. Aynı nesneye yönelik yapı doğrudan veri yapılarında da görülür.

Hedef şudur: Python'da listenin -1. elemanına erişirseniz son elemanı, -2. elemanına erişirseniz ondan öncekini verir. Java'da bunu yaparsak istisna oluşur. Diyelim algoritmamızda son elemanı bulmak için `main` içinde tek tek uğraşmak istemiyoruz. Bir kere "-1 gelirse şöyle ver" diye tanımlayan bir liste oluşturup onu kullanırız.

6.2 get Metodunu Override Etmek

`ArrayList` içinde `get` metodu vardır. Biz bunu `public E get(int index)` imzasıyla override ederiz. Dönüş tipi `E`, yani sınıf nasıl tanımlandıysa odur. Gövdeye yalnızca `super.get(index)` yazarsak davranış değişmez, yine çalışır. Negatif indeksler için şu fikir kullanılır: Gelen indeks sıfırdan küçükse listenin boyutuyla toplanır. Örneğin 10 elemanlı listede -3 gelirse $10+(-3) = 7$ olur (python ile hesaplandı). Yani gerçek indeks 7'dir.

Hoca önce elemanları kare olan bir liste kurdu. Karesini ilk yazışında yanlış yaptığını fark etti ve `i * i` olarak çarpmayla düzeltti. Liste şöyledir (python ile hesaplandı):

$$[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]$$

`get(3)` çağrısı 3'ün karesi olan 9'u yazdı. Normal `ArrayList` üzerinde `get(-1)` çağrısı hata verdi, çünkü Java'yı yazanlar bunu öyle tanımlamamış.

> **Ders sonrası not:** Hoca hatayı derste `ArrayIndexOutOfBoundsException` olarak gösterdi. JDK 25 ile denemede `ArrayList.get(-1)` şunu verdi: `IndexOutOfBoundsException: Index -1 out of bounds for length 10` (javac ile denendi). Bu fark büyük olasılıkla derste kullanılan JDK sürümündendir.

6.3 İlk Deneme Neden Çalışmadı?

Hoca ilk sürümde `if` içine `super.get(super.size() + index)` yazdı. Listeyi `MyArrayList` olarak tanımlayınca da aynı hata alındı.

```
// Çalışmayan ilk sürüm
@Override
```

```
public E get(int index) {
    if (index < 0)
        super.get(super.size() + index);  // Sonuç kullanılmıyor
    return super.get(index);              // index hâlâ negatif
}
```

Bu satır doğru elemanı okur, ama sonucu kullanmaz. Akış aşağı iner ve `super.get(index)` negatif indeksle çağrılır.

Soru: Bir öğrenci şunu önerdi: `super.get` kısmını silip `index = ...` yapsak sorun çözülmez mi? **Cevap:** Hoca "büyük ihtimalle şuraya `return` yazmak gerekiyordu, senin dediğin de doğru" dedi. Mevcut satır `return` demediği için akışın devam ettiğini açıkladı ve öğrencinin önerdiği gibi gelen indeksi değiştirdi. Tek satırlık `if` olduğu için süslü parantezlerin fark etmediğini de söyledi.

İki çözüm de javac ile denendi:

- `if` içinde `index = super.size() + index;` yazıp aşağıdaki `return` satırına inmek.
- `if` içinde `return super.get(super.size() + index);` yazmak.

İkisinde de `get(-1)` 81 verdi. İlk sürümde ise yine `IndexOutOfBoundsException` alındı. `get(-1)` sonucu 81, son eleman 9'un karesidir.

Hoca buradaki noktayı şöyle özetledi: `ArrayList` nasıl uygulanıyor, nasıl yazılmış hiçbir fikrim yok. Yalnızca `get` metodunu override ederek eksili elemana erişebiliyorum.

7. MyArrayList'e Silinen Elemanları Kaydetmek

Hoca örneği bir adım daha ilerletti. Listeye elemanlar ekliyoruz, siliyoruz, yine ekliyoruz, siliyoruz. Bir yandan da silinen elemanları takip etmek istiyoruz. Program çalıştı, bir hafta geçti ve neler silinmiş görmek istiyorum diye düşünün. Bunu da `extends` ettiğimiz listenin üstüne ekleyebiliriz.

Sınıfın içine ikinci bir `ArrayList` tanımlanır. Hoca bunu normalde kurucu metotta yapacağımızı, burada ise doğrudan alanda yaptığını söyledi. Sonra `remove` metodu override edilir: Silinen eleman, yeni listeye de eklenir. Hoca `remove` metodunun iki hali olduğunu belirtti: biri `int` alır, biri nesne alır. Silinenlere dışarıdan ulaşmak için bir de get metodu yazıldı.

```
import java.util.ArrayList;

class MyArrayList<E> extends ArrayList<E> {
    private ArrayList<E> removedItems = new ArrayList<>();

    @Override
    public E get(int index) {
        if (index < 0)
            index = super.size() + index;
        return super.get(index);
    }

    @Override
    public E remove(int index) {
        E silinen = super.remove(index);
        removedItems.add(silinen);
        return silinen;
    }

    public ArrayList<E> getRemovedItems() {
        return removedItems;
    }
}

public class Main2 {
```

```
    public static void main(String[] args) {
        MyArrayList<Integer> listem = new MyArrayList<>();
        for (int i = 0; i < 10; i++) listem.add(i * i);
        System.out.println(listem.get(3));
        System.out.println(listem.get(-1));

        listem.remove(3);
        listem.remove(5);
        for (int i = 0; i < 10; i++) listem.add(i * i);
        listem.remove(11);
        listem.remove(12);

        System.out.println(listem);
        System.out.println(listem.getRemovedItems());
    }
}
```

Çıktı (javac ile denendi, liste işlemleri python ile de hesaplandı):

```
9
81
[0, 1, 4, 16, 25, 49, 64, 81, 0, 1, 4, 16, 36, 49, 64, 81]
[9, 36, 9, 25]
```

Silme adımları adım adım şöyledir:

1. Başlangıç: $[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]$.
2. `remove(3)` 9'u siler. Liste: $[0, 1, 4, 16, 25, 36, 49, 64, 81]$.
3. `remove(5)` 36'yı siler. Liste: $[0, 1, 4, 16, 25, 49, 64, 81]$.
4. Yeni 10 kare eklenir, boyut 18 olur.
5. `remove(11)` ikinci turdaki 9'u siler (ara adım eklendi).
6. `remove(12)` kaymadan sonra o indekste kalan 25'i siler (ara adım eklendi).

`Integer` listesinde `remove(3)` çağrısı 3 değerini değil, 3. indeksteki elemanı siler. Bu yüzden silinenler listesinin başında 9 vardır. Silinenler, listenin kendisinden bağımsız ayrı bir yerde tutulur.

> **Ek bilgi (derste anlatılmadı):** Yukarıdaki sınıf yalnızca `remove(int)` metodunu override eder. `ArrayList` içindeki `remove(Object)` bu metodu çağırmaz: `listem.remove(Integer.valueOf(81))` elemanı siler, ama `removedItems` listesi boş kalır (javac ile denendi). Bu yüzden silinenlerin tamamı izlenecekse iki hal de override edilmelidir. Hocanın iki hale dikkat çekmesi bu açıdan önemlidir.

Hoca örneği şu sözlerle bağladı: Yalnızca nesneleri, insanları değil, veri yapılarını da nesneye yönelik olarak kullanabiliriz. Güzelliği şudur: `get` ya da `remove` metodunun içinin nasıl yazıldığını bilmiyorum, ama kendi ihtiyacıma göre veri yapısı oluşturabiliyorum. Bunu da var olanların üstünden `extends` diyerek çok kolay yapabilirim. Slaytta "Methods of Collection Interface" başlığıyla `Collection` arayüzünün metotları da vardı. İstersem doğrudan `implements Collection` deyip bu metotları kendim yazarak sıfırdan bir veri yapısı da oluşturabilirim.

Dersin Toparlaması

Bu uygulamada dört konu aynı düşünceyle bağlandı. Arayüz, ilgisiz sınıflara ortak bir sözleşme verir ve sınıfın içine bakmadan hangi metotların çağrılabileceğini söyler. `try`, `catch`, `finally` ve `throw`, sıfıra bölme ya da dizi sınırını aşma gibi durumlarda programın nasıl devam edeceğini belirler. Jenerikler aynı kodu farklı tiplerle kullanmayı sağlar. Koleksiyonlar ise bu kavramların hepsinin Java'nın kendi kütüphanesinde kullanıldığını gösterir: `List` bir arayüzdür, `ArrayList` jeneriktir, hatalı indeks istisna verir.

Dersin en önemli mesajı `MyArrayList` örneğindedir. `ArrayList` sınıfının içini bilmeden, yalnızca `get` ve `remove` metotlarını override ederek negatif indeks desteği ve silinenleri izleme özelliği eklendi. Yani kendi ihtiyacımıza göre veri yapısını, var olan bir yapının üstüne birkaç satırla kurabiliriz.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.